

NMHS Hospital Management Information System

Deployment, Hosting & System Administration Guide

SYSTEM	NMHS-HMIS-V1.0
DOCUMENT	NMHS-HMIS-V1.0-DG
VERSION	1.0
DATE	1 October 2026
AUDIENCE	Technical administrators and developers
REPOSITORY INSPECTED	main @ 1da43f3 + uncommitted Procedure work

Source of truth. This guide was written from the NMHS-HMIS-V1.0 repository as inspected on 1 October 2026 (branch `main`, commit `1da43f3`, plus the uncommitted working tree). Settings, dependencies, tasks and file locations were read from the code. Where the repository does not yet contain something a production deployment needs, this guide says so and labels any example as a **proposal**, not existing configuration. No secrets appear in this document; every credential is a placeholder.

Contents

Read this first: production configuration status	4
1. System architecture	4
2. Application requirements	5
2.1 Development versus production	6
3. Server requirements	6
4. Domain, DNS and network	6
5. Production environment variables	7
6. Database (PostgreSQL)	8
6.1 Why not SQLite in production	8
6.2 How PostgreSQL is selected	8
6.3 Creating the database	8
6.4 Migrations	8
6.5 Moving existing SQLite data (optional)	9
6.6 Maintenance	9
7. Redis and Celery	9
7.1 What runs in the background	9
7.2 Installing Redis	9
7.3 Running the worker and Beat	9
7.4 Health checks	10
8. WebSockets	10
9. Backend deployment	10
9.1 Gunicorn (systemd)	11
9.2 Celery worker and Beat (systemd)	11
9.3 Logs	11
10. Frontend deployment	12
11. Nginx reverse proxy	12
12. SSL / HTTPS	14
12.1 Django's HTTPS settings	14
13. Django production security	14
14. File storage and media	15
15. Backups	15
15.1 Verifying a backup	15
16. Monitoring	16
17. Logging	16
17.1 What is never logged	17

17.2 Audit trail and request IDs	17
17.3 Rotation and retention	17
18. Updates and maintenance	17
19. Disaster recovery	18
20. Production checklist	18
21. Troubleshooting	19
22. Security and operational warnings	20
23. Appendices	21
A. Useful commands	21
B. Directory structure	21
C. Service names (as proposed in this guide)	21
D. Health checks	21
E. Departments seeded by migrations	22
F. Support information	22

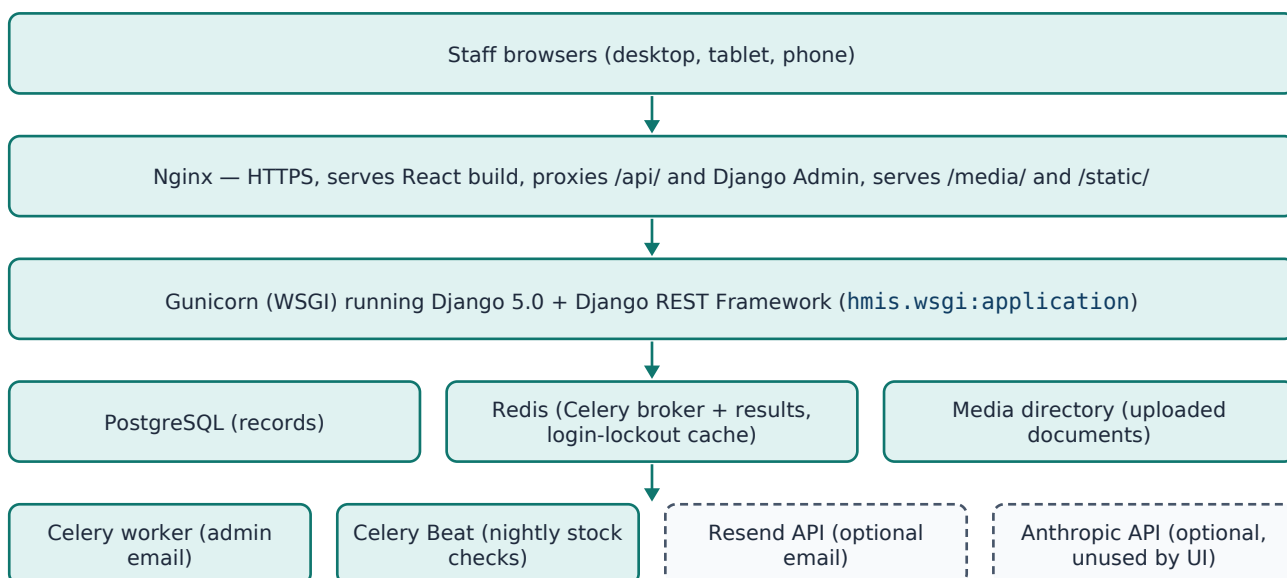
Read this first: production configuration status

The settings now distinguish **development** from **production** explicitly with `DJANGO_ENV` (`hmis/environment.py`). Development is unchanged: SQLite, `runserver`, nothing to configure. Production uses PostgreSQL only and **refuses to start** when a required variable is missing — there is no fallback to SQLite. `manage.py check --deploy` exits non-zero until the configuration is secure. Full detail is also in `docs/DEPLOYMENT.md` in the repository.

#	Item	Status in the repository
1	Database	Development: SQLite. Production: PostgreSQL from <code>DB_*</code> ; missing variables stop start-up (<code>ImproperlyConfigured</code>).
2	Static files	<code>STATIC_ROOT</code> set (<code>staticfiles/</code> , or <code>DJANGO_STATIC_ROOT</code>).
3	CSRF trusted origins	From <code>DJANGO_CSRF_TRUSTED_ORIGINS</code> ; production requires <code>https://</code> origins.
4	HTTPS settings	Production: HTTPS redirect, secure cookies, HSTS, proxy SSL header (each env-controlled, secure by default).
5	Password validators	Production: length 10+, common, numeric, similarity (<code>createsuperuser</code> and Django Admin forms).
6	DEBUG / SECRET_KEY	Production refuses <code>DJANGO_DEBUG=true</code> and placeholder or short keys.
7	<code>.env</code> files	Still not loaded by the application, deliberately: supply variables with <code>systemd EnvironmentFile=</code> .
8	Uploaded files (<code>/media/</code>)	Still no access control of their own — restrict in Nginx (chapter 14).
9	<code>/admin</code> path collision	Still present — serve Django Admin on its own hostname (chapter 11).
10	Tracked data files	<code>db.sqlite3.bak-20260903-073530</code> and <code>dump.rdb</code> are still in version control — review and remove (chapter 22).
11	API tokens	Do not expire; they end on sign-out and on a password reset.
12	Logging	Safe production logging with request IDs and redaction (chapter 17); health check at <code>/healthz/</code> .

1. System architecture

NMHS-HMIS-V1.0 is a Django REST API with a React single-page application. In production the browser loads the built React files from Nginx and calls the API on the **same origin** under `/api/`; the frontend uses the relative base URL `/api` (`frontend/src/api/client.js`).



Component	Required?	Role in NMHS-HMIS-V1.0
React + Vite build	Yes	The user interface. Static files only after <code>npm run build</code> .
Django + DRF (Gunicorn)	Yes	All business rules, permissions and data access. WSGI (<code>hmis/wsgi.py</code>); there is no <code>asgi.py</code> .
PostgreSQL	Yes for production	System of record. Development uses SQLite (see chapter 6).
Redis	Recommended	Celery broker and result backend (<code>REDIS_URL</code>); also the cache for the login lockout so failed-attempt counts are shared across Gunicorn workers.
Celery worker	Recommended	Sends administrator emails off the request (<code>apps.core.tasks.send_admin_email_task</code>). Without a broker, email is sent inline.
Celery Beat	Optional	Runs <code>check_low_stock</code> and <code>check_expiring_batches</code> every 24 hours. They compute lists only; they send no notification (chapter 7).
Nginx + TLS	Yes	Reverse proxy, HTTPS termination, static and media serving.
WebSockets	Not used	Live updates are polling (chapter 8).
Resend	Optional	Outbound HTTPS email for four admin events.
Anthropic API	Optional	<code>apps/ai_agents</code> exists but has no URL routes and no screen calls it.

2. Application requirements

Item	Repository fact	Production recommendation
Operating system	Not specified in the repository.	Ubuntu Server 24.04 LTS (or another current Linux LTS).
Python	Development venv runs Python 3.13.11. Django 5.0 officially supports Python 3.10–3.12.	Python 3.12 (the 24.04 system Python).
Django / DRF	Django==5.0.6, djangorestframework==3.15.1, django-filter==24.2, django-cors-headers==4.3.1	As pinned in <code>requirements.txt</code> .
Database driver	psycopg2-binary==2.9.10	As pinned.
PostgreSQL	Driver present; settings block commented out.	PostgreSQL 14 or newer (Django 5.0 requires 12+); 16 ships with Ubuntu 24.04.
Redis	redis==5.0.4 client; <code>REDIS_URL</code> default <code>redis://localhost:6379/0</code> .	Redis 7 from the OS packages, bound to localhost.
Celery	celery==5.4.0; app in <code>hmis/celery.py</code> .	Worker + Beat under systemd.
WSGI server	gunicorn==22.0.0 in requirements; no config file.	Gunicorn under systemd behind Nginx.
Node.js / npm	Vite 5 build; development machine Node 24.15.0 / npm 11.12.1.	Node 18+ (LTS 20 or 22) on the build machine only; the server needs only the built files.
Frontend	React 18, React Router 6, TanStack Query 5, Tailwind 3, Vite 5.	Build with <code>npm ci && npm run build</code> .
Other Python	anthropic==0.34.0, python-dotenv==1.0.1 (unused).	—

2.1 Development versus production

	Development (as in the repository)	Production (this guide)
Database	SQLite backend/hmis/db.sqlite3	PostgreSQL
Web server	python manage.py runserver on :8000	Gunicorn on 127.0.0.1:8000
Frontend	npm run dev (Vite, :5173) proxying /api and /media to :8000	Static build served by Nginx
DEBUG	True (default)	False
Media	Served by Django (DEBUG only)	Served by Nginx with access restriction
Email	Not sent when RESEND_API_KEY is empty	Optional, via Resend
HTTPS	None	Required

3. Server requirements

NOTE: Recommendations, not hard requirements

The project documentation states the system is built for about 50 concurrent users on a single server. The sizing below is a recommendation for that load; measure and adjust.

Resource	Minimum (pilot)	Recommended (≈50 concurrent users)
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB (Gunicorn ~5–9 workers, PostgreSQL, Redis, Celery)
System + application disk	40 GB SSD	80–100 GB SSD
PostgreSQL data	Inside the system disk	Separate volume, sized at 10× current data and monitored
Media (uploads)	Inside the system disk	Separate volume; grows with scanned results (about 6 MB in the development copy today)
Backup storage	Off-server	Off-server, encrypted, ≥ 30 days of daily dumps + media
Network	Hospital LAN	Static IP; 10 Mbit/s+ symmetrical if reached over the internet; UPS for server and network

Growth: clinical records are small; uploaded documents dominate disk growth. Review disk usage monthly (chapter 16) and scale RAM first if PostgreSQL begins to swap.

4. Domain, DNS and network

- Choose a hostname for staff, for example `hmis.example-hospital.org` (placeholder), and a second one for Django Admin, for example `hmis-admin.example-hospital.org` (chapter 11).
- Create DNS **A** records (and **AAAA** if the server has IPv6) pointing both names at the server's public IP.
- Internal-only option:** if the HMIS is used only inside the hospital, use internal DNS names resolving to the server's LAN address, and either an internal certificate authority or a public certificate obtained with a DNS challenge.
- Always use HTTPS. Patient data must never cross the network in plain HTTP.

5. Production environment variables

The application reads the process environment only (hmis/settings.py, hmis/environment.py). Put the values in a root-owned file such as /etc/nmhs-hmis/hmis.env (mode 600) and load it with systemd's `EnvironmentFile=`. Keep comments on their own lines — systemd does not strip a comment written after a value. backend/hmis/.env.example lists every variable with placeholders.

Variable	Purpose	Req.	Example / guidance
DJANGO_ENV	Selects the environment	Yes	production
DJANGO_SECRET_KEY	Signing key	Yes	50+ random characters; placeholders refused
DJANGO_DEBUG	Debug mode	No	Leave unset or False; true is refused
DJANGO_ALLOWED_HOSTS	Hostnames served	Yes	hmis.example-hospital.org,hmis-admin.example-hospital.org (no *)
DJANGO_CSRF_TRUSTED_ORIGINS	Origins allowed to POST forms	Yes	https://hmis.example-hospital.org,https://hmis-admin.example-hospital.org
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST	PostgreSQL connection	Yes	hmis / hmis / <strong-password> / 127.0.0.1
DB_PORT / DB_CONN_MAX_AGE	Port; connection reuse (s)	No	5432 / 60
REDIS_URL	Celery broker + results; lockout cache	Rec.	redis://127.0.0.1:6379/0
CACHE_URL	Overrides the cache location	No	redis://127.0.0.1:6379/1
DJANGO_BEHIND_HTTPS_PROXY / DJANGO_SECURE_SSL_REDIRECT	Proxy SSL header; HTTPS redirect	No	Both default true
DJANGO_SECURE_HSTS_SECONDS	HSTS max-age	No	Default 31536000
DJANGO_SECURE_HSTS_INCLUDE_SUBDOMAINS / _PRELOAD	HSTS scope	No	Default false (opt-in: commits the whole domain)
CORS_ALLOWED_ORIGINS	Cross-origin callers	No	Same-origin needs none
DJANGO_STATIC_ROOT / DJANGO_MEDIA_ROOT	File locations	No	Default staticfiles/, media/
DJANGO_LOG_LEVEL / DJANGO_LOG_DIR	Log level; optional rotating files	No	INFO / unset (journald)
DJANGO_LOG_MAX_BYTES / DJANGO_LOG_BACKUPS	Rotation size and count	No	10485760 / 10
DJANGO_REQUEST_LOG	One line per request	No	Default true in production
DJANGO_SQL_DEBUG	SQL logging	No	Development only; ignored in production
TIME_ZONE	Server time zone	No	Default Africa/Lagos
LOGIN_MAX_FAILED_ATTEMPTS / LOGIN_LOCKOUT_SECONDS	Lockout	No	5 / 300
RESEND_API_KEY / HMIS_EMAIL_FROM / HMIS_ADMIN_EMAIL / HMIS_BASE_URL	Optional admin email	No	<resend-api-key> / hmis@<domain> / ... / https://hmis.<domain>
ANTHROPIC_API_KEY	AI agents (unused by the UI)	No	Leave unset

WARNING: Secrets

Never put real keys or passwords in the repository, in tickets or in documentation. `.env` and `.env.*` are in `.gitignore` (`.env.example` is the only tracked one).

6. Database (PostgreSQL)

6.1 Why not SQLite in production

- SQLite allows one writer at a time; with ~50 staff saving vitals, payments and dispensing concurrently, writes queue and can time out.
- `select_for_update` row locks — used throughout dispensing, billing and admissions — are a documented no-op on SQLite. The code has database-independent safeguards for the critical races, but PostgreSQL provides real row locking as designed.
- PostgreSQL offers online backups (`pg_dump`), roles, and point-in-time recovery options.

6.2 How PostgreSQL is selected

With `DJANGO_ENV=production`, `DATABASES` is built from `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST` and `DB_PORT` (`hmis/environment.py`). If any of the first four is missing, start-up stops with a message naming the variables (never their values), for example:

```
django.core.exceptions.ImproperlyConfigured: Production uses PostgreSQL and these
variables are not set: DB_NAME, DB_USER, DB_PASSWORD, DB_HOST. There is no fallback to SQLite.
```

Development ignores these variables and keeps using `backend/hmis/db.sqlite3`.

6.3 Creating the database

```
sudo apt install postgresql
sudo -u postgres psql <<'SQL'
CREATE ROLE hmis LOGIN PASSWORD '<strong-password>';
CREATE DATABASE hmis OWNER hmis ENCODING 'UTF8';
REVOKE ALL ON DATABASE hmis FROM PUBLIC;
SQL
```

Keep PostgreSQL listening on localhost only (the default `listen_addresses`), and allow only the `hmis` role from `127.0.0.1` in `pg_hba.conf`.

6.4 Migrations

```
cd /srv/nmhs-hmis/backend/hmis
set -a; . /etc/nmhs-hmis/hmis.env; set +a
./venv/bin/python manage.py migrate
./venv/bin/python manage.py makemigrations --check --dry-run # must print "No changes detected"
```

Migrations also seed reference data: the revenue departments, Clinicals (Nursing), Maternity, **Procedure** (separate from **Theatre**), stock locations (Main Store, Pharmacy, returns-quarantine), item categories, the laboratory catalogue (66 tests), imaging and maternity price lists, and maternity visit types and options. Seeds never overwrite values an administrator has edited.

6.5 Moving existing SQLite data (optional)

IMPORTANT: Rehearse on a copy

Only needed if data entered on the development SQLite database must be kept. Because the migrations seed rows, loading a dump on top of a migrated database can conflict. Rehearse on a scratch PostgreSQL database, reconcile counts table by table, and run `manage.py billing_integrity` afterwards before relying on it.

```
# On the SQLite copy
./venv/bin/python manage.py dumpdata --natural-foreign --natural-primary \
-e contenttypes -e auth.permission -e sessions --indent 2 > hmis-data.json
# On the new PostgreSQL database (after migrate)
./venv/bin/python manage.py loaddata hmis-data.json
```

6.6 Maintenance

- Autovacuum is on by default; check `pg_stat_user_tables` for tables never vacuumed.
- Run `./venv/bin/python manage.py billing_integrity` weekly (read-only report of untraceable payments, patients in credit and ledger drift; `--strict` exits non-zero on findings).
- Keep PostgreSQL minor versions updated with the OS packages.

7. Redis and Celery

7.1 What runs in the background

Task	Trigger	What it does
<code>apps.core.tasks.send_admin_email_task</code>	On commit of patient registration, charge raised, payment received, discharge	Sends one admin email through Resend; retries 3 times with back-off; de-duplicated per event. If no broker accepts the job it is sent inline.
<code>apps.inventory.tasks.check_low_stock</code>	Celery Beat, every 24 h	Returns items below their reorder threshold. No notification is sent ; the result is stored in the result backend only.
<code>apps.inventory.tasks.check_expiring_batches</code>	Celery Beat, every 24 h	Returns batches expiring within the configured warning window, by location. No notification is sent.

Stock and expiry warnings that staff actually see are calculated live on the dashboards, independently of these tasks.

7.2 Installing Redis

```
sudo apt install redis-server
# /etc/redis/redis.conf: keep bind 127.0.0.1 ::1 and protected-mode yes
sudo systemctl enable --now redis-server
redis-cli ping          # PONG
```

7.3 Running the worker and Beat

The commands come from the project README; run them from `backend/hmis` so `-A hmis` resolves `hmis/celery.py`. systemd units are in chapter 9.

```
celery -A hmis worker -l info
celery -A hmis beat -l info
```

7.4 Health checks

```
cd /srv/nmhs-hmis/backend/hmis
./venv/bin/celery -A hmis inspect ping          # each worker answers "pong"
systemctl status nmhs-celery nmhs-celerybeat
journalctl -u nmhs-celery --since "1 hour ago"
```

8. WebSockets

NOTE: Not used by NMHS-HMIS-V1.0

The repository contains no WebSocket, Django Channels or ASGI code (there is no `asgi.py`). Nothing needs an ASGI server, a channel layer or WebSocket proxying.

Live information is refreshed by **polling** the REST API:

What	Interval	Where
Unread notification count (bell)	30 seconds and on window focus	<code>components/AppShell.jsx</code>
Triage (vitals) station queue	60 seconds	<code>pages/VitalsStation.jsx</code>
Department station queues (lab, ultrasound, eye, procedures)	60 seconds	<code>pages/DepartmentStation.jsx</code>

Production consideration: with 50 users the bell alone is about 100 small requests per minute — trivial for Gunicorn, but keep Nginx access logs rotated. If WebSockets are added in future, that will need ASGI, a channel layer and an Nginx `Upgrade` block; none of that exists today.

9. Backend deployment

Paths below (`/srv/nmhs-hmis`, user `hmis`) are placeholders; use your own conventions consistently.

1. Create a system user: `sudo adduser --system --group --home /srv/nmhs-hmis hmis`.
2. Clone the repository into `/srv/nmhs-hmis` (as `hmis`).
3. Create the virtual environment and install dependencies.
4. Create `/etc/nmhs-hmis/hmis.env` (chapter 5), owned by root, mode 600.
5. Set `DJANGO_ENV=production` and the required variables; nothing in the code needs editing.
6. Run migrations, collect static files, create the first administrator and run checks.
7. Install the systemd units and start the services.

```
sudo -u hmis git clone <repository-url> /srv/nmhs-hmis
cd /srv/nmhs-hmis/backend/hmis
sudo -u hmis python3.12 -m venv venv
sudo -u hmis ./venv/bin/pip install -r requirements.txt

# load the environment into this shell for manage.py
set -a; . /etc/nmhs-hmis/hmis.env; set +a
./venv/bin/python manage.py migrate
./venv/bin/python manage.py collectstatic --noinput
./venv/bin/python manage.py createsuperuser
./venv/bin/python manage.py check --deploy
./venv/bin/python manage.py makemigrations --check --dry-run
```

IMPORTANT: Give the first administrator a role

`createsuperuser` does not ask for the HMIS role, and the HMIS uses the **role** to decide access: a superuser with no role can open Django Admin but sees nothing in the HMIS. Sign in to Django Admin → Users, open the account and set **Role = Super Admin**.

9.1 Gunicorn (systemd)

```
# /etc/systemd/system/nmhs-gunicorn.service
[Unit]
Description=NMHS-HMIS-V1.0 Django (Gunicorn)
After=network.target postgresql.service redis-server.service

[Service]
User=hmis
Group=hmis
WorkingDirectory=/srv/nmhs-hmis/backend/hmis
EnvironmentFile=/etc/nmhs-hmis/hmis.env
ExecStart=/srv/nmhs-hmis/backend/hmis/venv/bin/gunicorn hmis.wsgi:application \
    --bind 127.0.0.1:8000 --workers 5 --timeout 60 --error-logfile -
# No --access-logfile: Django's hmis.request line replaces it, without query strings.
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

9.2 Celery worker and Beat (systemd)

```
# /etc/systemd/system/nmhs-celery.service
[Unit]
Description=NMHS-HMIS-V1.0 Celery worker
After=network.target redis-server.service

[Service]
User=hmis
Group=hmis
WorkingDirectory=/srv/nmhs-hmis/backend/hmis
EnvironmentFile=/etc/nmhs-hmis/hmis.env
ExecStart=/srv/nmhs-hmis/backend/hmis/venv/bin/celery -A hmis worker -l info
Restart=on-failure

[Install]
WantedBy=multi-user.target

# /etc/systemd/system/nmhs-celerybeat.service – same [Unit]/[Install], with:
ExecStart=/srv/nmhs-hmis/backend/hmis/venv/bin/celery -A hmis beat -l info \
    --schedule /srv/nmhs-hmis/celerybeat-schedule
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now nmhs-gunicorn nmhs-celery nmhs-celerybeat
```

9.3 Logs

Every service logs to stdout, so systemd's journal holds it: `journalctl -u nmhs-gunicorn -f`. See chapter 17 for what is and is not logged.

10. Frontend deployment

1. On a build machine with Node 18+: `cd frontend && npm ci && npm run build`.
2. The output is `frontend/dist/` (`index.html` and `assets/`). It is gitignored; build it for every release.
3. Copy `dist/` to the server, e.g. `/srv/nmhs-hmis/frontend/dist`.
4. No environment configuration is needed: the app calls `/api` on its own origin and links uploaded files at `/media/`. The hospital letterhead is inlined into the bundle.
5. Nginx serves `dist/` and must fall back to `index.html` for client-side routes such as `/patients/<uuid>`.

```
cd /srv/nmhs-hmis/frontend
npm ci
npm run build      # vite build -> dist/
npm test           # optional: the vitest suite
```

11. Nginx reverse proxy

Because the React app owns the path `/admin` (the HMIS Administration pages) and Django Admin owns `/admin/`, use **two server names**: one for staff (React + API) and one for Django Admin. Both proxy to the same Gunicorn. All names and paths are placeholders.

```
# /etc/nginx/sites-available/nmhs-hmis.conf
upstream nmhs_django { server 127.0.0.1:8000; }

server {
    # redirect HTTP -> HTTPS
    listen 80;
    server_name hmis.example-hospital.org hmis-admin.example-hospital.org;
    return 301 https://$host$request_uri;
}

server {
    # staff application
    listen 443 ssl http2;
    server_name hmis.example-hospital.org;
    ssl_certificate /etc/letsencrypt/live/hmis.example-hospital.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/hmis.example-hospital.org/privkey.pem;

    client_max_body_size 20m; # uploaded test results (adjust)
    add_header X-Content-Type-Options nosniff always;
    add_header Referrer-Policy same-origin always;
    add_header Strict-Transport-Security "max-age=31536000" always;

    root /srv/nmhs-hmis/frontend/dist;

    location /healthz/ { proxy_pass http://nmhs_django; proxy_set_header Host $host; }
    location /api/ {
        proxy_pass http://nmhs_django;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr; # overwrite: the lockout reads the first hop
        proxy_set_header X-Request-ID $request_id;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /media/ { # patient documents: restrict (chapter 14)
        alias /srv/nmhs-hmis/backend/hmis/media/;
        allow 10.0.0.0/8; # <hospital LAN range>
        deny all;
    }
    location /assets/ { expires 30d; access_log off; }
    location / { try_files $uri /index.html; }
}

server {
    # Django Admin (back office)
    listen 443 ssl http2;
    server_name hmis-admin.example-hospital.org;
    ssl_certificate /etc/letsencrypt/live/hmis-admin.example-hospital.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/hmis-admin.example-hospital.org/privkey.pem;
    allow 10.0.0.0/8; # <admin network>
    deny all;

    location /static/ { alias /srv/nmhs-hmis/backend/hmis/staticfiles/; } # STATIC_ROOT
    location / {
        proxy_pass http://nmhs_django;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr; # overwrite: the lockout reads the first hop
        proxy_set_header X-Request-ID $request_id;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

No WebSocket Upgrade headers are needed (chapter 8). Test with `sudo nginx -t` and `sudo systemctl reload nginx`.

12. SSL / HTTPS

- **Public names:** Let's Encrypt with certbot: `sudo certbot --nginx -d hmis.example-hospital.org -d hmis-admin.example-hospital.org`. Renewal runs from the certbot systemd timer; check with `sudo certbot renew --dry-run`.
- **Internal names:** an internal CA certificate installed on staff devices, or a public certificate obtained with a DNS-01 challenge.
- Redirect all HTTP to HTTPS (the port-80 server block above).
- Monitor expiry (chapter 16); an expired certificate stops every browser.

12.1 Django's HTTPS settings

In production these are on by default and each can be adjusted by environment variable: `SECURE_PROXY_SSL_HEADER` (Nginx's X-Forwarded-Proto), `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS` (one year; start lower while first testing HTTPS). `SECURE_HSTS_INCLUDE_SUBDOMAINS` and `SECURE_HSTS_PRELOAD` are opt-in. `/healthz/` is exempt from the redirect so a loopback probe reaches it. Nginx must send X-Forwarded-Proto \$scheme, or every request would redirect forever (and check `--deploy` reports `hmis.E007`).

13. Django production security

Control	Production behaviour
DEBUG	Off; <code>DJANGO_DEBUG=true</code> refused at start-up
SECRET_KEY	Required, 50+ characters, placeholders refused
ALLOWED_HOSTS	Required; * refused
CSRF trusted origins	Required; <code>https://</code> only
Cookies / HTTPS	Secure cookies, HTTP-only session cookie, HTTPS redirect, HSTS, proxy SSL header
Headers	<code>nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy: same-origin</code> , <code>Cross-Origin-Opener-Policy: same-origin</code> (all environments)
CORS	<code>CORS_ALLOWED_ORIGINS</code> ; a same-origin deployment needs none
Authentication	DRF Token + Session; tokens end on sign-out and password reset
Login brute force	5 failures → 5 min per client + username; share via Redis
Password policy	Validators on <code>createsuperuser</code> and Django Admin forms (not the HMIS Users screen's API — a separate decision)
Errors	500 answers a reference only; 409 a fixed sentence; validation errors stay readable
Uploads	Restrict <code>/media/</code> in Nginx; size limit <code>client_max_body_size</code>
Least privilege	Role groups in <code>accounts/permissions.py</code> ; every endpoint tested per role
Deployment gate	<code>check --deploy</code> errors <code>hmis.E001-E010</code> until all of the above hold

`./venv/bin/python manage.py check --deploy` must exit 0. The only remaining warnings are W005 / W021 (HSTS subdomains / preload) when left off deliberately.

14. File storage and media

Location	Contents	Backup
backend/hmis/media/medical_tests/YYYY/MM/	Uploaded results (images, PDFs, documents) on the Medical Tests & Diagnostics record and unit reports	Yes — daily, with the database
backend/hmis/media/investigations/YYYY/MM/	Attachments of the legacy diagnostics app (no screen uses it today)	Yes
STATIC_ROOT (e.g. staticfiles/)	Django Admin CSS/JS from collectstatic	No — regenerated
frontend/dist/	Built React app	No — rebuilt from source
Generated documents	None stored: every printout is rendered in the browser	Not applicable

WARNING: Uploaded patient documents have no access control of their own

The application links uploaded files by plain URL under `/media/`, and Django does not check permissions when they are served. Anyone with the URL who can reach the server can download the file. Restrict `/media/` to the hospital network in Nginx (as in chapter 11) or keep the whole service on the internal network.

15. Backups

What	How	Frequency / retention
PostgreSQL	<code>pg_dump -Fc</code> (custom format)	Daily; keep 30 daily, 12 monthly
Media	<code>tar</code> or <code>rsync</code> of <code>media/</code>	Daily (incremental), same retention
Configuration	<code>/etc/nmhs-hmis/</code> , Nginx and systemd unit files	On every change
Off-server copy	Encrypted copy to another site / provider	Daily
Restore test	Restore into a scratch database and server	Monthly

```
#!/bin/sh
# /usr/local/sbin/nmhs-backup (run daily from cron or a systemd timer as root)
set -eu
STAMP=$(date +%F)
DEST=/var/backups/nmhs/$STAMP
mkdir -p "$DEST"
sudo -u postgres pg_dump -Fc hmis > "$DEST/hmis.dump"
tar -C /srv/nmhs-hmis/backend/hmis -czf "$DEST/media.tar.gz" media
# encrypt before leaving the server (recipient key is a placeholder)
gpg --batch --yes --recipient <backup-key-id> --encrypt "$DEST/hmis.dump"
gpg --batch --yes --recipient <backup-key-id> --encrypt "$DEST/media.tar.gz"
rm "$DEST/hmis.dump" "$DEST/media.tar.gz"
# copy off-server (destination is a placeholder)
rsync -a /var/backups/nmhs/ <backup-user>@<backup-host>:/backups/nmhs/
find /var/backups/nmhs -maxdepth 1 -type d -mtime +30 -exec rm -rf {} +
```

15.1 Verifying a backup

```
gpg --decrypt hmis.dump.gpg > /tmp/hmis.dump
pg_restore --list /tmp/hmis.dump | head # the archive is readable
sudo -u postgres createdb hmis_restore_test
sudo -u postgres pg_restore -d hmis_restore_test /tmp/hmis.dump
sudo -u postgres psql hmis_restore_test -c "select count(*) from patients_patient;"
```

Compare a few counts (patients, charges, payments) with production, then drop the test database. Record the date and result of every restore test.

16. Monitoring

Component	What to watch	How
Server	CPU, RAM, swap, load	<code>htop</code> , <code>free -h</code> , <code>uptime</code> ; a monitoring agent
Disk	Root, PostgreSQL and media volumes > 80 %	<code>df -h</code> , <code>du -sh media</code>
Nginx	5xx rate, upstream errors	<code>/var/log/nginx/error.log</code> , access-log status codes
Django / Gunicorn	Service up, RF- error references	<code>systemctl status nmhs-gunicorn</code> , <code>journalctl -u nmhs-gunicorn</code>
PostgreSQL	Connections, long queries, size	<code>pg_stat_activity</code> , <code>pg_database_size('hmis')</code>
Redis	Up, memory	<code>redis-cli ping</code> , <code>redis-cli info memory</code>
Celery	Workers responding, failed email	<code>celery -A hmis inspect ping</code> , journal lines from logger <code>hmis.email</code>
TLS	Days to expiry	<code>sudo certbot certificates</code>
Application data	Ledger consistency	<code>manage.py billing_integrity weekly</code>
WebSockets	Not applicable — not used	—

An external uptime check can request <https://hmis.example-hospital.org/> (expects 200) and <https://hmis.example-hospital.org/healthz/> (expects **200** {"status": "healthy"}, or 503 when the database is unreachable).

17. Logging

Users see safe errors; administrators read logs; nothing in a log helps an attacker or becomes a second copy of patient records. All output goes to stdout, so under systemd the **journal** is the log (recommended). Every line carries a request ID: 2026-10-01 10:02:03 INFO hmis.request [3f2a...] GET /api/patients/ 200 41ms user=12.

Logger	Contents
<code>hmis.request</code>	Method, path, status, duration, user id — never the query string, headers, cookies or body
<code>hmis.api</code>	Unexpected errors with traceback and RF- reference; 409 conflicts (rule name only, never the conflicting values)
<code>hmis.security</code> , <code>django.security</code>	Sign-in success / failure / logout, sign-out, permission denials (user id, role, method, path, view)
<code>hmis.email</code>	Admin email outcomes (fixed subjects, hospital number only)
<code>hmis.health</code>	Health-check failures (exception class only)
<code>celery</code>	Worker and Beat through the same handlers — task name and id, never arguments
<code>django.db.backends</code>	WARNING only; SQL never logged in production

17.1 What is never logged

Passwords (including what was typed as a username on a failed sign-in), tokens, `Authorization` headers, cookies, CSRF tokens, request bodies, query strings, the secret key, database and Redis passwords, API keys, private keys — and patient records, diagnoses, results, prescriptions, notes, addresses, phone numbers or financial details. A `RedactingFilter` on every handler also scrubs credential shapes and the configured secret values from messages and tracebacks.

17.2 Audit trail and request IDs

- The existing database audit trail (`core.AuditLog`, Django Admin → Core → Audit logs) now also records `auth.login` and `auth.logout`. Failed sign-ins go to the security log only, so a guessing run cannot grow a table.
- X-Request-ID: a valid incoming value is kept, otherwise generated; returned on every response and carried to Celery tasks in a message header (Beat tasks use `task-<id>`). Nginx can pass its own with `proxy_set_header X-Request-ID $request_id;`.

17.3 Rotation and retention

Destination	How	Retention
journald (recommended)	<code>SystemMaxUse=2G, MaxRetentionSec=90day</code> in <code>/etc/systemd/journal.conf</code>	90 days
Files (optional)	<code>DJANGO_LOG_DIR</code> → <code>hmis.log</code> , <code>error.log</code> , <code>security.log</code> , each rotating at 10 MB × 10	Size-capped
Nginx	<code>logrotate</code> ; log format without <code>\$args/\$request</code> (searches travel in the query string)	14 days default

```
log_format hmis '$remote_addr - [$time_local] "$request_method $uri $server_protocol" '
                '$status $body_bytes_sent $request_time "$http_x_request_id"';
access_log /var/log/nginx/hmis.access.log hmis;
```

18. Updates and maintenance

- Backup** — run the backup script and confirm the dump and media archive exist.
- Verify the current version** — `git -C /srv/nmhs-hmis log -1 --oneline`; note it for rollback.
- Update code** — `git fetch && git checkout <release-tag-or-commit>`.
- Dependencies** — `./venv/bin/pip install -r requirements.txt`.
- Migrations** — `./venv/bin/python manage.py migrate` (review new migrations first; they are additive by project convention).
- Static files** — `./venv/bin/python manage.py collectstatic --noinput`.
- Frontend** — build `dist/` and replace the served copy.
- Restart** — `sudo systemctl restart nmhs-gunicorn nmhs-celery nmhs-celerybeat`.
- Health checks** — `manage.py check`, the two uptime URLs, `celery inspect ping`, sign in as a test user.
- Logs** — `watch journalctl -u nmhs-gunicorn -f` for RF- references for 15 minutes.
- .
- Rollback** — if needed, restore the previous commit and `dist/`; if a migration ran, restore the database from step 1 (do not attempt to reverse data migrations on a live system), then restart.
- .

IMPORTANT: After every model change

Always run `migrate` on the server. An unapplied migration does not fail loudly: the first screen that touches the new column returns an error and the page simply says it could not load.

19. Disaster recovery

Scenario	Recovery
Server failure	Build a new server (chapters 6-12), restore the latest database dump and media archive, restore <code>/etc/nmhs-hmis/</code> , point DNS at the new IP.
Database failure / corruption	Stop Gunicorn and Celery; create a fresh database; <code>pg_restore</code> the latest good dump; run <code>migrate</code> (no-op if the same version); start services; run <code>billing_integrity</code> .
Corrupted deployment	Check out the last known-good commit, rebuild <code>dist/</code> , reinstall requirements, restart.
Accidental deletion	Most clinical and financial records cannot be deleted by design. For anything lost, restore the dump into a scratch database and copy the rows back under developer supervision.
Restoring media	Extract <code>media.tar.gz</code> into <code>backend/hmis/</code> and fix ownership (<code>chown -R hmis:hmis media</code>).
DNS recovery	Keep registrar credentials in the hospital's secure store; record TTLs; re-point A/AAAA records.
SSL recovery	Re-issue with certbot on the new server (<code>certbot --nginx -d ...</code>).

Write down a recovery time objective and recovery point objective with hospital management, and rehearse the full rebuild at least once a year.

20. Production checklist

Before deployment

- Full backend and frontend suites run on the release being deployed.
- Server sized, OS patched, firewall allowing only 22 (restricted), 80 and 443.
- DNS names created; certificates obtainable.
- Env file written with `DJANGO_ENV=production`, unique secrets, hosts, https origins and `DB_*`.
- Backup destination and encryption key prepared.

During deployment

- `migrate` and `collectstatic` succeed
- `check --deploy` reviewed
- `makemigrations --check` reports no changes
- Superuser created **and given the Super Admin role**
- All three systemd services active

After deployment

- HTTPS works on both names; HTTP redirects
- `/healthz/` returns healthy
- Request lines with request IDs in the journal
- Django Admin has its CSS
- A test upload opens from the LAN and is refused from outside

- First backup run and restored to a scratch database

Before opening to hospital staff

- Departments, staff memberships (including **Procedure** staff), wards, beds, prices and hospital settings configured
- One account per person with the narrowest role
- Each role walks through its workflow with test patients
- Test patients purged (Super Admin, Django Admin) before go-live

Daily

- Backups succeeded
- Services active
- No new RF- errors
- Disk < 80 %

Weekly

- `billing_integrity`
- OS security updates
- Review failed-login lockouts
- Certificate expiry > 21 days

Monthly

- Restore test
- Review user accounts and roles
- Review disk growth
- Review Python/npm dependency advisories

21. Troubleshooting

Symptom	Likely cause	Fix
502 Bad Gateway	Gunicorn down or not on 127.0.0.1:8000	<code>systemctl status nmhs-gunicorn</code> ; read the journal; check env file and venv path.
Frontend cannot reach API	Nginx <code>/api/</code> location missing; API on another origin	Proxy <code>/api/</code> on the same server name; the app always calls <code>/api</code> .
Page refresh gives 404	No SPA fallback	<code>try_files \$uri /index.html</code> ;
WebSocket connection failure	Not applicable	NMHS-HMIS-V1.0 uses no WebSockets; look for a proxy or browser extension.
Celery tasks not running	Worker/Beat stopped, wrong directory, broker down	<code>systemctl status nmhs-celery</code> ; <code>celery -A hmis inspect ping</code> from backend/hmis.
Redis unavailable	Service stopped or bound elsewhere	<code>redis-cli ping</code> . Login lockout fails open and email falls back to inline sending, so staff keep working.
Database connection failure	Wrong DB_*, PostgreSQL down, SQLite block still active	Check DB_*; <code>psql -h 127.0.0.1 -U hmis hmis; /healthz/</code> says unhealthy until fixed.
ImproperlyConfigured at start-up	A required production variable is missing or unsafe	The message names it; set it in the env file and restart.
Migration failure	Unapplied or conflicting migrations	Restore from backup if partially applied; never edit applied migrations.

Symptom	Likely cause	Fix
Static files missing (Django Admin unstyled)	No <code>STATIC_ROOT</code> / <code>collectstatic</code>	Run <code>collectstatic</code> ; <code>/static/</code> alias to <code>STATIC_ROOT</code> on the admin server name.
Media files missing	Wrong alias, permissions, or <code>DEBUG</code> -only serving assumed	Nginx alias path; <code>chown hmis</code> ; restrict, don't remove, the location.
Login fails / CSRF error in Django Admin	Origin not in <code>CSRF_TRUSTED_ORIGINS</code> ; HTTPS not recognised	Add the origin to <code>DJANGO_CSRF_TRUSTED_ORIGINS</code> ; send <code>X-Forwarded-Proto</code> from Nginx.
"Too many attempts" on login	Lockout after 5 failures	Wait 5 minutes or clear the user's cache entry; never disable the lockout.
CORS errors	Frontend served from a different origin	Serve same-origin, or add the origin to <code>CORS_ALLOWED_ORIGINS</code> .
Slow server	Swap, too few/many workers, large tables	Check RAM, <code>pg_stat_activity</code> , Unicorn workers $\approx 2 \times \text{CPU} + 1$.
Disk full	Media, logs, backups kept locally	Clean old local backups, rotate logs, grow the media volume.
SSL certificate problems	Expired or wrong name	<code>certbot certificates</code> , <code>certbot renew</code> , reload Nginx.
User sees nothing after sign-in	Account has no HMIS role (e.g. fresh superuser)	Set the role in Django Admin → Users.

22. Security and operational warnings

- **Never commit `.env` files or secrets.** `.env` is gitignored; keep secrets in `/etc/nmhs-hmis/hmis.env` (mode 600).
- **Repository data files:** `backend/hmis/db.sqlite3.bak-20260903-073530` and `backend/hmis/dump.rdb` are committed. If they contain real patient data they must be removed from version control *and its history*, and treated as a data-protection incident. Never copy them to a server.
- Never expose PostgreSQL (5432) or Redis (6379) to the internet; bind them to localhost.
- Use strong, unique administrator passwords; keep Django Admin on its own restricted host.
- Restrict SSH: keys only, no root login, source-IP limits where possible.
- Firewall: allow 443 (and 80 for redirect / certificate renewal), restricted 22; deny the rest.
- Keep the OS, Python packages and npm packages updated; rebuild after updates.
- **Back up before every migration** and test restores monthly.
- Protect patient data: HTTPS only, restricted `/media/`, sign out on shared PCs, deactivate leavers the same day.
- Limit administrative access: few Super Admins; use Hospital Admin for day-to-day administration; review accounts quarterly.

23. Appendices

A. Useful commands

```
# backend (from /srv/nmhs-hmis/backend/hmis, env loaded)
./venv/bin/python manage.py check [--deploy]
./venv/bin/python manage.py migrate
./venv/bin/python manage.py makemigrations --check --dry-run
./venv/bin/python manage.py showmigrations
./venv/bin/python manage.py collectstatic --noinput
./venv/bin/python manage.py createsuperuser
./venv/bin/python manage.py billing_integrity [--strict]
./venv/bin/python manage.py seed_lab_catalogue      # re-runnable; never overwrites edits
./venv/bin/python manage.py test                  # full backend suite (long-running)
./venv/bin/celery -A hmis inspect ping

# frontend
npm ci && npm run build && npm test

# services
sudo systemctl status|restart nmhs-gunicorn nmhs-celery nmhs-celerybeat nginx redis-server postgresql
```

B. Directory structure

```
NMHS-HMIS-V1.0 repository
├── backend/hmis/
│   ├── manage.py
│   ├── requirements.txt
│   ├── .env.example
│   ├── hmis/
│   │   ├── apps/
│   │   │   ├── placeholders only; values come from the environment
│   │   │   ├── settings.py, urls.py, wsgi.py, celery.py, pagination.py
│   │   │   ├── accounts, patients, clinical, inventory, pharmacy, sales,
│   │   │   ├── appointments, ai_agents, core, departments, workflow, billing,
│   │   │   ├── diagnostics, laboratory, inpatient, maternity
│   │   │   └── uploaded files (gitignored)
│   └── media/
├── frontend/
│   ├── package.json, vite.config.js, tailwind.config.js
│   ├── src/
│   │   └── # React application
│   └── dist/
│       └── # build output (gitignored)
```

C. Service names (as proposed in this guide)

Service	Purpose
nginx	HTTPS, static, media, reverse proxy
nmhs-gunicorn	Django API and Django Admin
nmhs-celery	Celery worker
nmhs-celerybeat	Celery Beat scheduler
postgresql	Database
redis-server	Broker, results, lockout cache

D. Health checks

Check	Expected
curl -sI https://hmis.example-hospital.org/	200
curl -s https://hmis.example-hospital.org/healthz/	{"status": "healthy"}
redis-cli ping	PONG
celery -A hmis inspect ping	pong from each worker

Check	Expected
<code>sudo -u postgres psql -c 'select 1' hmis</code>	1
<code>manage.py check</code>	System check identified no issues

E. Departments seeded by migrations

Reception, Consultation, Laboratory, Pharmacy, Radiology / Ultrasound, Eye, **Theatre**, **Procedure**, Clinicals (Nursing) and Maternity. **Procedure and Theatre are separate departments** (migration `departments/0007` split the old combined row): each has its own staff list and its own revenue column, and membership of one grants nothing in the other. Procedure staff are assigned in Django Admin → Departments → Procedure.

F. Support information

Record here, for your installation: the hospital's HMIS owner, the technical administrator on call, the hosting provider and its support contact, the domain registrar, and where the backup encryption key and recovery credentials are stored. Keep this page in the hospital's secure documentation, not in the repository.